

Lab 06: Introduction to OMNeT++ and INET

Duration: 1 hour 30 minutes | In-Lab Assessment

Tools: OMNeT++ and INET Framework

Objectives:

- Understand what OMNeT++ and INET are and why they are used.
- Learn basic OMNeT++ components: NED files, modules, gates and connections.
- Run a prebuilt INET example in the OMNeT++ IDE (QtEnv) and inspect results.
- Create and run a very small wired Ethernet network (2 hosts + switch) using INET, and verify connectivity with PingApp.

Short theory:

What is OMNeT++?

OMNeT++ is a discrete event simulation environment used to model communication networks, queuing networks and distributed systems. It provides an IDE (graphical) where you can write simple network descriptions (NED), implement behaviour in C++ if needed, run the simulation in a visual environment (QtEnv) and collect results.

What is INET?

INET is a large open source model library/framework built on OMNeT++. It provides pre-built, realistic protocol stacks (Ethernet, IP, TCP/UDP, wireless PHY/MAC, routing, lots of applications). Using INET saves you from implementing low level protocols and lets you focus on experiments. (See INET documentation and examples for details.)

Key OMNeT++ concepts:

- **NED files:** describe the network topology (modules and how they connect).
- **Modules:** building blocks. *Simple module* contains behaviour (in C++). *Compound module* contains submodules and connections.
- **Gates and connections:** modules talk through gates, connected by channels (wired links).
- **omnetpp.ini:** main configuration file (which network to run, parameter values, runtime choices).
- **QtEnv:** the graphical runtime where you can see packet animations. INET examples are prebuilt to use this. Wired link channels typically use a DatarateChannel with datarate/delay parameters.

Before the lab

- OMNeT++ installed (use official installer for your OS).
- INET framework downloaded and built inside OMNeT++ (import INET into the IDE and build; many INET examples appear in the Project Explorer).

- Familiarize: File → New → OMNeT++ Project, and how to open `omnetpp.ini` and NED files.
- If you have trouble installing, follow the documentation provided in the classroom

Part A: Run a ready-made INET example

1. Open OMNeT++ IDE.
2. In the Project Explorer open the INET examples folder (e.g. `inet/examples/ethernet/lans`). You should see NED networks and `omnetpp.ini` files. The repository's examples are a good starting point.
3. Right-click on an example's `omnetpp.ini` (or network NED) and choose `Run As → OMNeT++ Simulation` (or use Run Configurations).
4. In Qtenv you will see nodes and links. Press the green play button to let the simulation run (packets animate) or use step controls to single-step events.
5. Open the console/Status window — INET apps such as `PingApp` print ping results (RTT, packet loss). The simulation writes scalar/vector output files into the result directory specified in `omnetpp.ini`.

Part B: .ned file Explanation

File: Networks.ned

```

1 //
2 // Copyright (C) 2003 CTIE, Monash University
3 //
4 // SPDX-License-Identifier: LGPL-3.0-or-later
5 //
6 //
7
8 package inet.examples.ethernet.lans;
9
10 import inet.node.ethernet.EthernetHub;
11 import inet.node.ethernet.Eth100M;
12 import inet.node.ethernet.Eth10M;
13 import inet.node.ethernet.EthernetHost;
14 import inet.node.ethernet.EthernetSwitch;
15 import inet.physicallayer.wired.common.WireJunction;
16 import ned.DatarateChannel;
17
18 //
19 // Sample Ethernet LAN containing eight hosts, a switch and a bus.
20 //
21 network MixedLAN
22 {
23     types:
24         channel C extends Eth10M
25         {
26             length = 1cm;
27         }

```

```
28 submodules:
29     tap1: WireJunction {
30         parameters:
31             @display("p=126,307");
32     }
33     tap2: WireJunction {
34         parameters:
35             @display("p=258,307");
36     }
37     tap3: WireJunction {
38         parameters:
39             @display("p=344,307");
40     }
41     tap4: WireJunction {
42         parameters:
43             @display("p=514,307");
44     }
45     tap5: WireJunction {
46         parameters:
47             @display("p=650,307");
48     }
49     tap6: WireJunction {
50         parameters:
51             @display("p=753,307");
52     }
53     busHostA: EthernetHost {
54         parameters:
55             eth.duplexMode = false;
56             @display("p=126,200");
57     }
58     busHostB: EthernetHost {
59         parameters:
60             eth.duplexMode = false;
61             @display("p=258,200");
62     }
63     busHostC: EthernetHost {
64         parameters:
65             eth.duplexMode = false;
66             @display("p=514,400");
67     }
68     busHostD: EthernetHost {
69         parameters:
70             eth.duplexMode = false;
71             @display("p=753,400");
72     }
73     switchHostA: EthernetHost {
74         parameters:
75             csmacdSupport = false;
76             eth.duplexMode = true;
77             @display("p=100,500");
78     }
79     switchHostB: EthernetHost {
80         parameters:
```

```

81         csmacdSupport = false;
82         eth.duplexMode = true;
83         @display("p=250,500");
84     }
85     switchHostC: EthernetHost {
86         parameters:
87             csmacdSupport = false;
88             eth.duplexMode = true;
89             @display("p=400,500");
90     }
91     switchHostD: EthernetHost {
92         parameters:
93             csmacdSupport = false;
94             eth.duplexMode = true;
95             @display("p=550,500");
96     }
97     switch: EthernetSwitch {
98         parameters:
99             eth[4].duplexMode = false;
100             @display("p=344,400");
101         gates:
102             ethg[5];
103     }
104     hubHostA: EthernetHost {
105         parameters:
106             eth.duplexMode = false;
107             @display("p=500,100");
108     }
109     hubHostB: EthernetHost {
110         parameters:
111             eth.duplexMode = false;
112             @display("p=650,100");
113     }
114     hubHostC: EthernetHost {
115         parameters:
116             eth.duplexMode = false;
117             @display("p=800,100");
118     }
119     hub: EthernetHub {
120         parameters:
121             @display("p=650,200");
122     }
123     connections:
124         tap1.port++ <--> Eth10M { length = 10m; } <--> tap2.port++;
125         tap2.port++ <--> Eth10M { length = 10m; } <--> tap3.port++;
126         tap3.port++ <--> Eth10M { length = 4m; } <--> tap4.port++;
127         tap4.port++ <--> Eth10M { length = 11m; } <--> tap5.port++;
128         tap5.port++ <--> Eth10M { length = 8m; } <--> tap6.port++;
129
130     //half-duplex:
131     tap1.port++ <--> C <--> busHostA.ethg;
132     tap2.port++ <--> C <--> busHostB.ethg;
133     tap3.port++ <--> C <--> switch.ethg[4];

```

```

134     tap4.port++ <--> C <--> busHostC.ethg;
135     tap5.port++ <--> C <--> hub.ethg++;
136     tap6.port++ <--> C <--> busHostD.ethg;
137
138     //full-duplex:
139     switch.ethg[0] <--> Eth100M { length = 3m; } <--> switchHostA
140         .ethg;
141     switch.ethg[1] <--> Eth100M { length = 2m; } <--> switchHostB
142         .ethg;
143     switch.ethg[2] <--> Eth10M { length = 4m; } <--> switchHostC.
144         ethg;
145     switch.ethg[3] <--> Eth100M { length = 5m; } <--> switchHostD
146         .ethg;
147
148     //half-duplex:
149     hub.ethg++ <--> Eth10M { length = 3m; } <--> hubHostA.ethg;
150     hub.ethg++ <--> Eth10M { length = 4m; } <--> hubHostB.ethg;
151     hub.ethg++ <--> Eth10M { length = 5m; } <--> hubHostC.ethg;
152 }
153
154 //
155 // Sample Ethernet LAN: two hosts directly connected to each other
156 // via twisted pair.
157 //
158 network TwoHosts
159 {
160     submodules:
161         hostA: EthernetHost {
162             parameters:
163                 @display("p=100,100");
164         }
165         hostB: EthernetHost {
166             parameters:
167                 @display("p=300,100");
168         }
169     connections:
170         hostA.ethg <--> { delay = 0.5us; datarate = 100Mbps; } <-->
171             hostB.ethg;
172 }
173
174 //
175 // Sample Ethernet LAN: four hosts connected to a switch.
176 //
177 network SwitchedLAN
178 {
179     types:
180         channel C extends DatarateChannel
181         {
182             delay = 0.1us;
183             datarate = 100Mbps;
184         }

```

```

182     submodules:
183         hostA: EthernetHost {
184             parameters:
185                 csmacdSupport = false;
186                 eth.duplexMode = true;
187                 @display("p=250,100");
188         }
189         hostB: EthernetHost {
190             parameters:
191                 csmacdSupport = false;
192                 eth.duplexMode = true;
193                 @display("p=400,200");
194         }
195         hostC: EthernetHost {
196             parameters:
197                 csmacdSupport = false;
198                 eth.duplexMode = true;
199                 @display("p=250,300");
200         }
201         hostD: EthernetHost {
202             parameters:
203                 csmacdSupport = false;
204                 eth.duplexMode = true;
205                 @display("p=100,200");
206         }
207         switch: EthernetSwitch {
208             parameters:
209                 @display("p=250,200");
210             gates:
211                 ethg[4];
212         }
213     connections:
214         switch.ethg[0] <--> C <--> hostA.ethg;
215         switch.ethg[1] <--> C <--> hostB.ethg;
216         switch.ethg[2] <--> C <--> hostC.ethg;
217         switch.ethg[3] <--> C <--> hostD.ethg;
218 }
219
220
221 //
222 // Sample Ethernet LAN: four hosts connected by a hub.
223 //
224 network HubLAN
225 {
226     submodules:
227         hostA: EthernetHost {
228             parameters:
229                 eth.duplexMode = false;
230                 @display("p=250,100");
231         }
232         hostB: EthernetHost {
233             parameters:
234                 eth.duplexMode = false;

```

```

235         @display("p=400,200");
236     }
237     hostC: EthernetHost {
238         parameters:
239             eth.duplexMode = false;
240             @display("p=250,300");
241     }
242     hostD: EthernetHost {
243         parameters:
244             eth.duplexMode = false;
245             @display("p=100,200");
246     }
247     hub: EthernetHub {
248         parameters:
249             @display("p=250,200");
250         gates:
251             ethg[4];
252     }
253     connections:
254         hub.ethg[0] <--> { delay = 0.1us; datarate = 100Mbps; } <-->
255             hostA.ethg;
256         hub.ethg[1] <--> { delay = 0.3us; datarate = 100Mbps; } <-->
257             hostB.ethg;
258         hub.ethg[2] <--> { delay = 0.4us; datarate = 100Mbps; } <-->
259             hostC.ethg;
260         hub.ethg[3] <--> { delay = 0.2us; datarate = 100Mbps; } <-->
261             hostD.ethg;
262 }
263
264 //
265 // Sample Ethernet LAN: four hosts on a bus.
266 //
267 network BusLAN
268 {
269     types:
270         channel C extends Eth10M
271         {
272             length = 1cm;
273         }
274     submodules:
275         hostA: EthernetHost {
276             parameters:
277                 eth.duplexMode = false;
278                 @display("p=100,111");
279         }
280         hostB: EthernetHost {
281             parameters:
282                 eth.duplexMode = false;
283                 @display("p=250,111");
284         }
285         hostC: EthernetHost {
286             parameters:

```

```

284         eth.duplexMode = false;
285         @display("p=400,111");
286     }
287     hostD: EthernetHost {
288         parameters:
289             eth.duplexMode = false;
290             @display("p=550,111");
291     }
292     tap1: WireJunction {
293         parameters:
294             @display("p=100,180");
295     }
296     tap2: WireJunction {
297         parameters:
298             @display("p=250,180");
299     }
300     tap3: WireJunction {
301         parameters:
302             @display("p=400,180");
303     }
304     tap4: WireJunction {
305         parameters:
306             @display("p=550,180");
307     }
308     connections:
309         tap1.port++ <--> Eth10M <--> tap2.port++;
310         tap2.port++ <--> Eth10M <--> tap3.port++;
311         tap3.port++ <--> Eth10M <--> tap4.port++;
312         hostA.ethg <--> C <--> tap1.port++;
313         hostB.ethg <--> C <--> tap2.port++;
314         hostC.ethg <--> C <--> tap3.port++;
315         hostD.ethg <--> C <--> tap4.port++;
316 }
317
318 network SwitchedDuplexLAN
319 {
320     types:
321         channel ethernetline extends DatarateChannel
322         {
323             parameters:
324                 delay = 0.1us;
325                 datarate = 10Mbps;
326         }
327
328     submodules:
329         hostA: EthernetHost {
330             parameters:
331                 csmacdSupport = false;
332                 eth.duplexMode = true;
333                 @display("p=250,100");
334         }
335         hostB: EthernetHost {
336             parameters:

```



```

337         csmacdSupport = false;
338         eth.duplexMode = true;
339         @display("p=400,200");
340     }
341     hostC: EthernetHost {
342         parameters:
343             csmacdSupport = false;
344             eth.duplexMode = true;
345             @display("p=250,300");
346     }
347     hostD: EthernetHost {
348         parameters:
349             csmacdSupport = false;
350             eth.duplexMode = true;
351             @display("p=100,200");
352     }
353     switch: EthernetSwitch {
354         parameters:
355             @display("p=250,200");
356         gates:
357             ethg[4];
358     }
359     connections:
360         switch.ethg[0] <--> ethernetline <--> hostA.ethg;
361         switch.ethg[1] <--> ethernetline <--> hostB.ethg;
362         switch.ethg[2] <--> ethernetline <--> hostC.ethg;
363         switch.ethg[3] <--> ethernetline <--> hostD.ethg;
364 }

```

Explanation of important parts in Networks.ned

package & import lines

- `package inet.examples.ethernet.lans;` groups the NED definitions under a package name. This helps the IDE locate networks and avoids name clashes.
- `import inet.node.ethernet.EthernetHost;` etc. bring INET module types into scope so we can use them as submodules. These imported names (like `EthernetHost`) are prebuilt modules from INET.

network MixedLAN and types

- `types: channel C extends Eth10M { length = 1cm; }` declares a custom channel type named `C` which inherits properties from the INET `Eth10M` channel. You can then reuse `C` in many connections.
- Channels encapsulate physical link properties such as `datarate`, `delay` or `length`. These affect packet timing and throughput in simulations.

submodules: hosts, switch, hub, junctions

- `EthernetHost` is a prebuilt compound module representing a full host (NIC, protocol

stack, apps). Parameters like `eth.duplexMode = false` set the NIC to half-duplex (typical for bus/hub) or true for full-duplex (switch ports).

- `csmacdSupport` controls whether CSMA/CD behavior (carrier sense / collision detection) is used. For switched host links you often set it to `false` because switches avoid collisions.
- `EthernetSwitch` represents a layer-2 switch. The `gates: ethg[5];` declaration creates an array of Ethernet ports named `ethg` with 5 ports (indices 0..4).
- `WireJunction` is used for bus-style wiring / taps (it behaves like wires on a shared medium).
- `@display("p=x,y")` gives the coordinates for the graphical layout in Qtenv (purely visual).

connections: syntax and examples

- `hostA.ethg <-> delay = 0.5us; datarate = 100Mbps; <-> hostB.ethg;` connects two gates with an inline channel that sets delay and datarate.
- When you see `tap1.port++ <-> Eth10M length = 10m; <-> tap2.port++;`, the `port++` syntax tells NED to use the next available gate of a vector (useful when the junction has many ports).
- The operator `<->` denotes a bidirectional connection (symmetric).
- Comments such as `//half-duplex:` indicate the developer's intent: those connections model a shared-medium bus (half-duplex) while others model full-duplex switch links.

Multiple networks in one file

- This single NED file defines several example networks (`MixedLAN`, `TwoHosts`, `SwitchedLAN`, etc.). When you run a simulation you pick which network to instantiate from `omnetpp.ini`

Part C: A Simple Network Explanation

File: `Networks.ned/SwitchedLAN`

```

1  network SwitchedLAN
2  {
3      types:
4          channel C extends DatarateChannel
5          {
6              delay = 0.1us;
7              datarate = 100Mbps;
8          }
9      submodules:
10         hostA: EthernetHost {
11             parameters:
12                 csmacdSupport = false;
13                 eth.duplexMode = true;
14                 @display("p=250,100");
15         }
16         hostB: EthernetHost {
17             parameters:

```

```

18         csmacdSupport = false;
19         eth.duplexMode = true;
20         @display("p=400,200");
21     }
22     hostC: EthernetHost {
23         parameters:
24             csmacdSupport = false;
25             eth.duplexMode = true;
26             @display("p=250,300");
27     }
28     hostD: EthernetHost {
29         parameters:
30             csmacdSupport = false;
31             eth.duplexMode = true;
32             @display("p=100,200");
33     }
34     switch: EthernetSwitch {
35         parameters:
36             @display("p=250,200");
37         gates:
38             ethg[4];
39     }
40     connections:
41         switch.ethg[0] <--> C <--> hostA.ethg;
42         switch.ethg[1] <--> C <--> hostB.ethg;
43         switch.ethg[2] <--> C <--> hostC.ethg;
44         switch.ethg[3] <--> C <--> hostD.ethg;
45 }

```

SwitchedLAN Network Definition

```

1 network SwitchedLAN
2 {

```

This line declares a network named **SwitchedLAN**. It represents a switched Local Area Network where multiple Ethernet hosts communicate through a central switch.

Custom Channel Definition

```

1     types:
2         channel C extends DatarateChannel
3         {
4             delay = 0.1us;
5             datarate = 100Mbps;
6         }

```

Channel Type C The custom channel **C** extends **DatarateChannel**, allowing explicit configuration of delay and bandwidth for Ethernet links.

Delay The propagation delay is set to 0.1 microseconds, modeling a short-distance LAN cable.

Data Rate The data rate is configured as 100 Mbps, which corresponds to Fast Ethernet.

Submodules Description

```

1  submodules:
2      hostA: EthernetHost {
3          parameters:
4              csmacdSupport = false;
5              eth.duplexMode = true;
6              @display("p=250,100");
7      }

```

Ethernet Host A `hostA` is an instance of `EthernetHost`. It represents an end device connected to the LAN.

CSMA/CD Support `csmacdSupport = false` disables collision detection, which is unnecessary in a switched, full-duplex Ethernet network.

Duplex Mode `eth.duplexMode = true` enables full-duplex communication, allowing simultaneous transmission and reception.

Display Annotation The `@display` parameter specifies the visual position of the host in the OMNeT++ GUI.

Additional Ethernet Hosts

```

1      hostB: EthernetHost {
2          parameters:
3              csmacdSupport = false;
4              eth.duplexMode = true;
5              @display("p=400,200");
6      }
7      hostC: EthernetHost {
8          parameters:
9              csmacdSupport = false;
10             eth.duplexMode = true;
11             @display("p=250,300");
12     }
13     hostD: EthernetHost {
14         parameters:
15             csmacdSupport = false;
16             eth.duplexMode = true;
17             @display("p=100,200");
18     }

```

Each of these hosts (`hostB`, `hostC`, and `hostD`) is configured identically to `hostA`. All hosts operate in full-duplex mode and are visually positioned around the switch.

Ethernet Switch Configuration

```

1      switch: EthernetSwitch {
2          parameters:
3              @display("p=250,200");
4          gates:
5              ethg[4];
6      }

```

Switch Module The `EthernetSwitch` acts as the central forwarding device in the network.

Gate Array The switch defines four Ethernet gates (`ethg[4]`), each corresponding to a physical switch port used to connect one host.

Display Position The display annotation places the switch at the center of the topology in the simulation environment.

Connections Description

```

1 connections:
2     switch.ethg[0] <--> C <--> hostA.ethg;
3     switch.ethg[1] <--> C <--> hostB.ethg;
4     switch.ethg[2] <--> C <--> hostC.ethg;
5     switch.ethg[3] <--> C <--> hostD.ethg;
6 }
```

Host-to-Switch Links Each host is connected to a unique switch port using the custom channel `C`. The bidirectional operator (`<->`) ensures symmetric communication.

Dedicated Links Since each host has its own point-to-point link to the switch, no medium is shared among hosts.

Collision-Free Communication Because the links are full-duplex and point-to-point, collisions cannot occur, resulting in higher throughput and predictable performance.

Part D: .ini File Explanation

File: `omnetpp.ini`

```

1 [General]
2 sim-time-limit = 10s
3 **.vector-recording = false
4 description = "(abstract)"
5
6 [Config SimpleLan1]
7 network = SwitchedLAN
8 **.hostA.cli.destAddress = ""
9 **.cli.destAddress = "hostA"
10 **.cli.sendInterval = exponential(1s)
11 **.cli.reqLength = intuniform(50,1400)*1B
12 **.cli.respLength = intWithUnit(truncnormal(3000B,3000B))
13 **.switch*.bridging.typename = "MacRelayUnit"
```

Line-by-line explanation

```

1 [General]
2 sim-time-limit = 10s
```

Stops the simulation at 10 simulated seconds.

```

1 **.vector-recording = false
```

Globally disables vector recording (time-series vectors). You can selectively override this for specific modules.

```
1 description = "(abstract)"
```

A human-readable description; no effect on simulation logic.

```
1 [Config SimpleLan1]
2 network = SwitchedLAN
```

Defines a configuration named `SimpleLan1` and selects the `SwitchedLAN` network (the NED provided earlier).

```
1 **.hostA.cli.destAddress = ""
```

Sets the `destAddress` parameter of the `cli` submodule inside `hostA` to an empty string (likely means "not set").

```
1 **.cli.destAddress = "hostA"
```

Sets `destAddress` for all modules named `cli` to `"hostA"`. Note specificity/precedence rules: more specific keys (e.g., `**.hostA.cli.*`) override less specific.

```
1 **.cli.sendInterval = exponential(1s)
```

Sets the send interval distribution for all `cli` modules to an exponential distribution with mean 1s.

```
1 **.cli.reqLength = intuniform(50,1400)*1B
```

Request length is a uniformly random integer between 50 and 1400 bytes.

```
1 **.cli.respLength = intWithUnit(truncnormal(3000B,3000B))
```

Response length drawn from a truncated normal distribution with mean=3000B and std-dev=3000B — this is a very wide spread; truncated to avoid negative sizes. `intWithUnit` converts to integer unit.

```
1 **.switch*.bridging.typeName = "MacRelayUnit"
```

Sets the bridging implementation (typeName) used by switch modules that match the wildcard. Implementation names and exact parameter keys depend on the INET version.

Rules & patterns to remember

- Wildcards: `**` matches any module path; `*` matches one level. Use them to target many modules quickly.
- Specificity: a parameter targeted at `**.hostA.cli.param` is more specific than `**.cli.param` and will override it.
- Units: always include units for sizes and times (e.g., B, s, ms, bps).
- Check module names: verify that modules (like `cli`) and parameters (`reqLength`) exist in your INET version.

Concrete modifications and extensions

Below are ready-to-use INI lines. Put them under the appropriate [Config ...] section.

1) Change sender and destination

```
1 # Make hostA send to hostB
2 **.hostA.cli.destAddress = "hostB"
3
4 # Make hostB send to hostC; hostC -> hostD
5 **.hostB.cli.destAddress = "hostC"
6 **.hostC.cli.destAddress = "hostD"
7
8 # Set all CLI modules to send to hostD
9 **.cli.destAddress = "hostD"
```

2) Change send rate / inter-send distribution

```
1 # Deterministic 0.2s interval for hostA
2 **.hostA.cli.sendInterval = 0.2s
3
4 # Poisson-like with mean 0.5s for hostB
5 **.hostB.cli.sendInterval = exponential(0.5s)
6
7 # Different distributions per host
8 **.hostA.cli.sendInterval = uniform(0.1s,0.5s)
9 **.hostB.cli.sendInterval = exponential(1s)
10 **.hostC.cli.sendInterval = normal(0.5s,0.1s) # be careful with
    negatives
```

3) Change packet sizes

```
1 # Fixed request size 500 bytes
2 **.cli.reqLength = 500B
3
4 # Uniform request size between 100 and 1500 bytes
5 **.cli.reqLength = intuniform(100,1500)*1B
6
7 # Response size: truncated normal mean 2000B stddev 200B
8 **.cli.respLength = intWithUnit(truncnormal(2000B,200B))
9
10 # Deterministic response size
11 **.cli.respLength = 1200B
```

4) Start/stop times (stagger traffic)

```
1 # Start and stop times for hostA
2 **.hostA.cli.startTime = 0.5s
3 **.hostA.cli.stopTime = 8s
4
5 # Staggered starts
6 **.hostB.cli.startTime = 1s
7 **.hostC.cli.startTime = 2s
```

5) Multiple apps per host (example pattern)

```
1 # If your host supports multiple apps, configure them per-host
2 **.hostA.numApps = 2
3 **.hostA.app[0].typename = "UdpBasicApp"
```

```

4 **.hostA.app[0].destAddresses = "hostB"
5 **.hostA.app[0].sendInterval = 0.1s
6
7 **.hostA.app[1].typename = "UdpBasicApp"
8 **.hostA.app[1].destAddresses = "hostC"
9 **.hostA.app[1].sendInterval = 0.5s

```

(Replace app, numApps, and parameter names with the ones in your INET version.)

6) Change switch behavior

```

1 # Use a different bridge implementation (check your INET)
2 **.switch*.bridging.typename = "LearningBridge"
3
4 # Example MAC aging time (name may differ by INET)
5 **.switch*.bridging.macEntryTimeout = 300s

```

7) Enable recording selectively

```

1 # Global off, but enable for hostA and the switch
2 **.vector-recording = false
3 **.hostA*.vector-recording = true
4 **.switch*.vector-recording = true
5
6 # Or enable globally
7 **.vector-recording = true

```

8) Enable pcap / packet capture (INET-specific)

```

1 # Example conceptual enabling of pcap; exact parameter path depends on
  INET version
2 **.host*.ppp.interface[0].pcapRecorder.enable = true

```

(Inspect module tree in OMNeT++ GUI to find exact recorder module name.)

9) Repeatable runs / RNG seeds

```

1 # Repeatable experiments
2 seed-set = 42
3 repeat = 5

```

10) Per-link bandwidth (if channel parameters are addressable)

```

1 # Only valid if channel instances/parameters are accessible via INI
2 **.C.datarate = 1Gbps
3 **.C.delay = 1us

```

(If this does not work, define multiple channel types inside NED and use them there.)

11) Addressing and auto-configurator

```

1 # Use INET configurator features (parameter names depend on INET)
2 *.configurator.assignAddresses = true
3 *.configurator.addStaticRoutes = true

```

12) Emulate link failure (conceptual)

```

1 # You can schedule start/stop or write a control module that toggles
  gates.
2 # Alternatively, use apps' startTime/stopTime to emulate outages.
3 **.hostA.cli.stopTime = 5s # hostA stops sending at 5s

```


Example scenarios

A. HostA → HostB, high-rate small packets

```

1 **.hostA.cli.destAddress = "hostB"
2 **.hostA.cli.sendInterval = 0.01s
3 **.hostA.cli.reqLength = intuniform(64,128)*1B
4 **.hostA.cli.respLength = 128B

```

B. Many sporadic large responses to hostA

```

1 **.cli.destAddress = "hostA"
2 **.cli.sendInterval = exponential(2s)
3 **.cli.reqLength = 100B
4 **.cli.respLength = intWithUnit(truncnormal(3000B,500B))

```

C. Turn on vector recording only for hostA and the switch

```

1 **.vector-recording = false
2 **.hostA*.vector-recording = true
3 **.switch*.vector-recording = true

```

Troubleshooting tips & best practices

- If `destAddress` is empty, the app may not send. Always set explicit destinations or use broadcast where appropriate.
- Verify module and parameter names for your INET version — names vary across versions.
- Use the OMNeT++ inspector to view full module tree and available parameters.
- Keep units explicit (B, s, ms, bps).
- For reproducibility, set `seed-set` and `repeat`.
- When using distributions like `normal(...)` or `truncnormal(...)`, ensure stddev is reasonable to avoid degenerate behavior.
- If switching bridging implementations, confirm the targeted `typename` exists in your INET release.

Part E: GUI Usage

The core logic stays the same, but instead of coding you can drag and drop different components, modules, submodules etc. Will be shown during the lab.

Debugging tips

- **No packets visible/instant simulation end:** check `sim-time-limit` in `omnetpp.ini` and ensure `startTime` of apps is within limit.
- **IP not assigned / hosts cannot ping:** include an `Ipv4NetworkConfigurator` in NED or use `flatNetworkConfigurator` to set IPs automatically.
- **Wrong import or module name:** INET versions move module classes between packages. If an import fails, search for the module name in your INET `src` tree (IDE search).
- **Build errors:** ensure INET is built first and your project references it (Project → Properties → Project References).